

Morgan Stanley Banking Data Analysis Project

Objective

The objective of this project is to analyze customer transactions, account balances, risk scores, and credit ratings to identify business insights and support better financial decision-making.

▼

Task 1: Data Cleaning

In this task, the data is checked for missing values, duplicate records, and correct data types. The TransactionDate column is converted into date format so the data is ready for analysis.

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import pandas as pd

df = pd.read_csv('/content/drive/MyDrive/morgan_stanely.csv')

df.head()
```

	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	TransactionAmount
0	78	CUST1223	ACC33287	Credit	Withdrawal	Savings Account	Firm E	South	Manager 4	06-01-2023	81300.425190
1	21	CUST8266	ACC58667	Savings	Withdrawal	Credit Card	Firm A	South	Manager 2	17-02-2023	9269.640373
2	176	CUST9420	ACC99117	Credit	Transfer	Home Loan	Firm A	East	Manager 1	13-09-2023	28138.552650
3	167	CUST5253	ACC10117	Loan	Payment	Mutual Fund	Firm D	Central	Manager 1	28-05-2024	83943.556980
4	46	CUST1223	ACC74631	Savings	Deposit	Credit Card	Firm A	East	Manager 4	12-05-2023	77104.456470

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
# Dataset Shape

print("Rows and Columns:", df.shape)
```

Rows and Columns: (800, 15)

```
# Column Names

df.columns

Index(['TransactionID', 'CustomerID', 'AccountID', 'AccountType',
      'TransactionType', 'Product', 'Firm', 'Region', 'Manager',
      'TransactionDate', 'TransactionAmount', 'AccountBalance', 'RiskScore',
      'CreditRating', 'TenureMonths'],
      dtype='object')
```

```
# Dataset Information

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   TransactionID          800 non-null   int64
1   CustomerID             800 non-null   object
2   AccountID              800 non-null   object
```

```
3  AccountType      800 non-null object
4  TransactionType  800 non-null object
5  Product          800 non-null object
6  Firm             800 non-null object
7  Region           800 non-null object
8  Manager          800 non-null object
9  TransactionDate  800 non-null object
10 TransactionAmount 800 non-null float64
11 AccountBalance   800 non-null float64
12 RiskScore        800 non-null float64
13 CreditRating     800 non-null int64
14 TenureMonths     800 non-null int64
dtypes: float64(3), int64(3), object(9)
memory usage: 93.9+ KB
```

Dataset Information Analysis

Observation

- The dataset contains 800 records and 15 columns.
- No missing values are present in any column.
- Data types are appropriate for financial analysis.

Business Insight

ince there are no missing values and the data is well organized, the analysis results are more reliable.

Recommendation

The dataset is ready for analysis, so the bank can use it to study customer behavior and financial risk with confidence.

Statistical Summary

df.describe()

	TransactionID	TransactionAmount	AccountBalance	RiskScore	CreditRating	TenureMonths	
count	800.000000	800.000000	800.000000	800.000000	800.000000	800.000000	
mean	99.786250	55194.889273	72764.355699	0.473380	568.526250	119.870000	
std	57.460793	28940.337792	35623.574024	0.239982	155.620111	67.356806	
min	1.000000	-36173.695550	-62548.722060	-0.300118	306.000000	6.000000	
25%	50.000000	34842.164247	48684.199043	0.312271	437.000000	63.000000	
50%	99.500000	53768.493325	72908.053070	0.485829	565.000000	118.000000	
75%	150.000000	74729.551358	95574.572673	0.632799	699.500000	178.000000	
max	199.000000	154662.045800	180539.287700	1.254704	848.000000	238.000000	

Statistical Summary Analysis

Observation

- The dataset contains 800 transaction records.
- The average transaction amount is approximately 55,195.
- The average account balance is approximately 72,764.
- The average customer risk score is around 0.47.
- Customer tenure ranges from 6 to 238 months.

Business Insight

Customers have a good average account balance, while transaction amounts differ from customer to customer. The risk score shows that the dataset includes both low-risk and high-risk customers, making it useful for financial risk analysis.

Recommendation

The bank should regularly review customer risk scores and transaction patterns to identify high-risk customers and provide better financial services.

Check Missing Values

df.isnull().sum()

	0
TransactionID	0
CustomerID	0
AccountID	0
AccountType	0
TransactionType	0
Product	0
Firm	0
Region	0
Manager	0
TransactionDate	0
TransactionAmount	0
AccountBalance	0
RiskScore	0
CreditRating	0
TenureMonths	0

dtype: int64

```
# Check Duplicate Records
```

```
print("Duplicate Records:", df.duplicated().sum())
```

Duplicate Records: 0

df.dtypes

	0
TransactionID	int64
CustomerID	object
AccountID	object
AccountType	object
TransactionType	object
Product	object
Firm	object
Region	object
Manager	object
TransactionDate	object
TransactionAmount	float64
AccountBalance	float64
RiskScore	float64
CreditRating	int64
TenureMonths	int64

dtype: object

▼ Data Type Analysis

Observation: The dataset contains integer, float, and text data types.

```
# Convert TransactionDate to Date Format
```

```
df["TransactionDate"] = pd.to_datetime(df["TransactionDate"], dayfirst=True)
```

```
df["TransactionDate"].head()
```

	TransactionDate
0	2023-01-06
1	2023-02-17
2	2023-09-13
3	2024-05-28
4	2023-05-12

dtype: datetime64[ns]

Business Insight: The available data types are suitable for analysis. After converting the TransactionDate column into date format, time-based analysis can be performed easily.

Recommendation

Always convert date columns before performing time-based analysis.

Overall Business Insight:
The dataset is clean and complete. There are no missing values or duplicate records, and the data is ready for accurate financial analysis.

Overall Recommendation:
Always check data quality before analysis. Clean and accurate data leads to better analysis and better business decisions.

Task 2: Descriptive Transactional Analysis

In this task, the transaction data is analyzed to understand customer activity. We find monthly and yearly summaries, compare credit and debit transactions, and identify top, bottom, and inactive accounts.

```
# Total Transaction Amount

print("Total Transaction Amount:", df["TransactionAmount"].sum())

Total Transaction Amount: 44155911.4185837
```

Observation: The total transaction amount is 44,155,911.42, which represents the total value of all customer transactions in the dataset.

Business Insight: This helps the bank understand the total business volume and overall financial activity.

Recommendation
Regularly monitor the total transaction amount to understand business performance and identify changes in customer activity.

```
# Average Transaction Amount

print("Average Transaction Amount:", df["TransactionAmount"].mean())

Average Transaction Amount: 55194.88927322962
```

Recommendation
Regularly monitor the total transaction amount to understand business performance and identify changes in customer activity.

Observation: The average transaction amount is approximately 55,195.

Business Insight

Customers make transactions of around 55,195 on average. This shows the typical transaction value in the dataset.

Recommendation
Monitor the average transaction amount regularly to understand changes in customer transaction behavior.

```
# Create Month and Year Columns

df["Month"] = df["TransactionDate"].dt.month_name()
df["Year"] = df["TransactionDate"].dt.year

df[["TransactionDate", "Month", "Year"]].head()
```

	TransactionDate	Month	Year
0	2023-01-06	January	2023
1	2023-02-17	February	2023
2	2023-09-13	September	2023
3	2024-05-28	May	2024
4	2023-05-12	May	2023



Month and Year Analysis

Observation: The Month and Year columns were created from the TransactionDate column.

Business Insight: These new columns make it easier to analyze transaction trends by month and year.

Recommendation : Use the Month and Year columns to compare monthly and yearly transaction performance.

```
df["TransactionType"].value_counts()
```

	count
TransactionType	
Deposit	205
Withdrawal	203
Payment	197
Transfer	195

dtype: int64

Monthly Summary of Transaction Amount

```
monthly_summary = df.pivot_table(
    index="Month",
    columns="TransactionType",
    values="TransactionAmount",
    aggfunc="sum",
    fill_value=0
)
```

monthly_summary

TransactionType	Deposit	Payment	Transfer	Withdrawal
Month				
April	1.196326e+06	1.313353e+06	1.071101e+06	1.426692e+06
August	4.737014e+05	4.329217e+05	3.693893e+05	5.034818e+05
December	5.407030e+05	6.323292e+05	4.262729e+05	4.672820e+05
February	1.231897e+06	1.161034e+06	1.601820e+06	1.277157e+06
January	1.321995e+06	9.040777e+05	1.254937e+06	1.186558e+06
July	7.760727e+05	5.895807e+05	7.402434e+05	3.769130e+05
June	7.402068e+05	1.411667e+06	8.603178e+05	6.851719e+05
March	1.142629e+06	9.084730e+05	1.353500e+06	1.319019e+06
May	1.205283e+06	1.351174e+06	1.126601e+06	1.283023e+06
November	5.292799e+05	9.669516e+05	7.238952e+05	9.910534e+05
October	9.547712e+05	7.826544e+05	8.913282e+05	9.320519e+05
September	8.639821e+05	6.013822e+05	5.981983e+05	6.574582e+05



Next steps:

[Generate code with monthly_summary](#)

[New interactive sheet](#)

Monthly Transaction Summary

Observation: The table shows the total transaction amount for each month. The transaction amount changes from month to month.

Business Insight: Customer transaction activity is not the same every month. Some months have higher transaction amounts, while others have lower amounts.

Recommendation The bank should review the months with lower transaction amounts and plan offers or campaigns to increase customer activity.

Yearly Summary of Transaction Amount

```
yearly_summary = df.pivot_table(
    index="Year",
    columns="TransactionType",
    values="TransactionAmount",
    aggfunc="sum",
    fill_value=0
)

yearly_summary
```

TransactionType	Deposit	Payment	Transfer	Withdrawal
Year				
2023	7.727269e+06	7.719796e+06	7.488798e+06	7.806984e+06
2024	3.249579e+06	3.335802e+06	3.528806e+06	3.298878e+06



Next steps: [Generate code with yearly_summary](#) [New interactive sheet](#)

Obsevation

The total transaction amount is higher in 2023 than in 2024 across all transaction types (Deposit, Payment, Transfer, and Withdrawal).

Business Insight

Customer transaction activity was higher in 2023. This may indicate that more transactions were recorded in 2023 than in 2024.

Recommendation

Compare the same time period for both years before making business decisions. This helps ensure a fair comparison.

Total Credit, Debit and Net Transaction Volume

```
total_credit = df[df["TransactionType"] == "Deposit"]["TransactionAmount"].sum()

total_debit = df[df["TransactionType"] == "Withdrawal"]["TransactionAmount"].sum()

net_transaction = total_credit - total_debit

print("Total Credit:", round(total_credit, 2))
print("Total Debit:", round(total_debit, 2))
print("Net Transaction Volume:", round(net_transaction, 2))
```

Total Credit: 10976847.54
Total Debit: 11105861.38
Net Transaction Volume: -129013.83

Observation Total Credit is 10,976,847.54. Total Debit is 11,105,861.38. The Net Transaction Volume is -129,013.83, which means total debit is slightly higher than total credit.

Business Insight

Customers withdrew slightly more money than they deposited. This resulted in a small negative net transaction volume.

Recommendation

The bank should encourage more deposits through savings plans or fixed deposit offers to improve the overall transaction balance.

Account Type Summary

```
account_summary = df.groupby("AccountType")["TransactionAmount"].sum()

account_summary
```

	TransactionAmount
AccountType	
Credit	1.179965e+07
Current	1.003690e+07
Loan	1.102955e+07
Savings	1.128981e+07
dtype: float64	

Observation Credit accounts have the highest total transaction amount (11.80 million approximately). Current accounts have the lowest total transaction amount (10.04 million approximately). Savings and Loan accounts have similar transaction amounts.

Business Insight

Credit accounts show the highest transaction activity, while Current accounts have the lowest. This indicates that customers use Credit accounts more frequently for transactions.

Recommendation

The bank should maintain strong services for Credit account customers and encourage more transactions in Current accounts through suitable offers and banking services.

```
# Region-wise Transaction Summary

region_summary = df.groupby("Region")["TransactionAmount"].sum()

region_summary
```

	TransactionAmount
Region	
Central	9.743647e+06
East	8.334617e+06
North	8.356205e+06
South	8.900243e+06
West	8.821200e+06
dtype: float64	

Observation

The Central region has the highest total transaction amount (9.74 million approximately). The East region has the lowest total transaction amount (8.33 million approximately). The transaction amounts across all regions are fairly close.

Business Insight

The Central region shows the highest banking activity, while the East region has slightly lower transaction activity. Overall, transaction activity is balanced across all regions.

Recommendation

Continue providing strong banking services in the Central region and introduce customer engagement programs in the East region to increase transaction activity.

```
# Product-wise Transaction Summary

product_summary = df.groupby("Product")["TransactionAmount"].sum()

product_summary
```

	TransactionAmount
Product	
Credit Card	8.800657e+06
Home Loan	8.513817e+06
Mutual Fund	8.730287e+06
Personal Loan	8.336879e+06
Savings Account	9.774272e+06
dtype: float64	

Observation

Savings Account has the highest transaction amount, while Personal Loan has the lowest.

Business Insight

Customers made more transactions through Savings Accounts than Personal Loans.

Recommendation

Continue improving Savings Account services and promote Personal Loan products to increase transactions.

```
# Monthly Credit vs Debit Trend

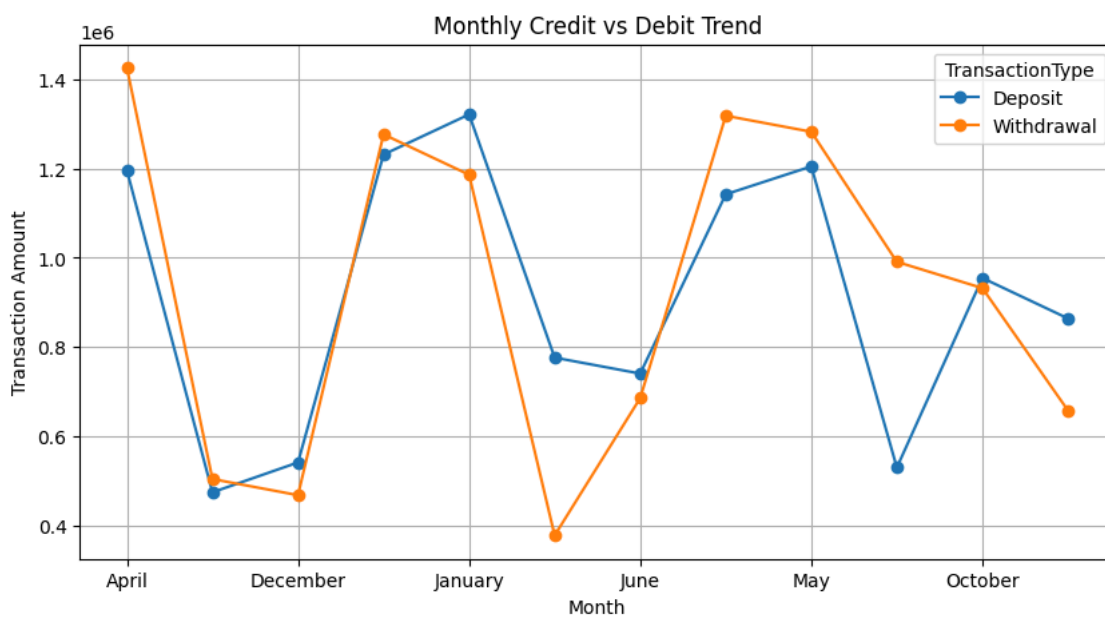
credit_debit = df[df["TransactionType"].isin(["Deposit", "Withdrawal"])]

monthly_trend = credit_debit.groupby(["Month", "TransactionType"])["TransactionAmount"].sum().unstack()

monthly_trend.plot(kind="line", figsize=(10,5), marker="o")

plt.title("Monthly Credit vs Debit Trend")
plt.xlabel("Month")
plt.ylabel("Transaction Amount")
plt.grid(True)

plt.show()
```



Observation

Deposit and Withdrawal amounts change every month. In some months, Withdrawals are higher, while in others, Deposits are higher.

Business Insight

Customer transaction activity changes from month to month.

Recommendation

Monitor monthly Deposit and Withdrawal trends to maintain a healthy transaction balance.

```
# Top and Bottom Performing Accounts

account_summary = df[df["TransactionType"].isin(["Deposit", "Withdrawal"])]

account_summary = account_summary.pivot_table(
    index="AccountID",
    columns="TransactionType",
    values="TransactionAmount",
    aggfunc="sum",
    fill_value=0
)

account_summary["NetInflow"] = account_summary["Deposit"] - account_summary["Withdrawal"]

print("Top 5 Performing Accounts")
display(account_summary.sort_values("NetInflow", ascending=False).head())

print("Bottom 5 Performing Accounts")
display(account_summary.sort_values("NetInflow").head())
```

Top 5 Performing Accounts

TransactionType	Deposit	Withdrawal	NetInflow
AccountID			
ACC48501	323846.55886	25215.946120	298630.612740
ACC57700	250121.85850	0.000000	250121.858500
ACC38559	219919.51623	24073.272256	195846.243974
ACC83581	184645.18445	0.000000	184645.184450
ACC18177	174360.26978	0.000000	174360.269780

Bottom 5 Performing Accounts

TransactionType	Deposit	Withdrawal	NetInflow
AccountID			
ACC33287	34882.50354	341262.80314	-306380.29960
ACC77592	0.00000	264162.85439	-264162.85439
ACC24070	0.00000	232635.75639	-232635.75639
ACC19156	0.00000	190330.38927	-190330.38927
ACC11285	0.00000	183035.94173	-183035.94173

Observation

Top 5 accounts have positive net inflow, and Bottom 5 accounts have negative net inflow.

Business Insight

Transaction performance differs across accounts.

Recommendation

Monitor low-performing accounts and encourage more deposits.

```
# Find Dormant / Inactive Accounts

df = df.sort_values(["AccountID", "TransactionDate"])

df["GapDays"] = df.groupby("AccountID")["TransactionDate"].diff().dt.days

dormant_accounts = df[df["GapDays"] >= 60][["AccountID", "TransactionDate", "GapDays"]]

print("Dormant / Inactive Accounts")
dormant_accounts
```

Dormant / Inactive Accounts

AccountID	TransactionDate	GapDays
555 ACC10117	2023-04-30	90.0
753 ACC10117	2024-02-05	128.0
3 ACC10117	2024-05-28	100.0
137 ACC10996	2023-08-17	116.0
515 ACC10996	2023-10-26	70.0
...	...	...
113 ACC96868	2024-06-11	66.0
134 ACC97411	2023-11-25	323.0
201 ACC99117	2024-05-22	252.0
444 ACC99409	2024-03-14	409.0
563 ACC99549	2024-02-04	200.0

326 rows × 3 columns

Next steps:

[Generate code with dormant_accounts](#)

[New interactive sheet](#)

```
# Add Status

df["Status"] = "Active"
df.loc[df["GapDays"] >= 60, "Status"] = "Dormant"

df[["AccountID", "TransactionDate", "GapDays", "Status"]].head(20)
```

	AccountID	TransactionDate	GapDays	Status	
58	ACC10117	2023-01-30	NaN	Active	
555	ACC10117	2023-04-30	90.0	Dormant	
509	ACC10117	2023-06-20	51.0	Active	
696	ACC10117	2023-07-14	24.0	Active	
727	ACC10117	2023-08-04	21.0	Active	
318	ACC10117	2023-09-30	57.0	Active	
753	ACC10117	2024-02-05	128.0	Dormant	
179	ACC10117	2024-02-18	13.0	Active	
3	ACC10117	2024-05-28	100.0	Dormant	
655	ACC10996	2023-02-05	NaN	Active	
98	ACC10996	2023-02-17	12.0	Active	
613	ACC10996	2023-03-16	27.0	Active	
659	ACC10996	2023-04-23	38.0	Active	
137	ACC10996	2023-08-17	116.0	Dormant	
515	ACC10996	2023-10-26	70.0	Dormant	
429	ACC10996	2023-11-19	24.0	Active	
78	ACC10996	2024-02-18	91.0	Dormant	
649	ACC11062	2023-07-19	NaN	Active	
13	ACC11062	2023-09-30	73.0	Dormant	
330	ACC11062	2023-12-25	86.0	Dormant	

```
# Dormant / Inactive Accounts

df = df.sort_values(["AccountID", "TransactionDate"])

df["PreviousDate"] = df.groupby("AccountID")["TransactionDate"].shift(1)

df["GapDays"] = (df["TransactionDate"] - df["PreviousDate"]).dt.days

dormant_accounts = df[df["GapDays"] >= 60][["AccountID", "GapDays"]]

print("Dormant / Inactive Accounts")
dormant_accounts.head()
```

	AccountID	GapDays	
555	ACC10117	90.0	
753	ACC10117	128.0	
3	ACC10117	100.0	
137	ACC10996	116.0	
515	ACC10996	70.0	

Next steps:

[Generate code with dormant_accounts](#)

[New interactive sheet](#)

✖ Dormant / Inactive Accounts

Observation:

The analysis identifies accounts with a transaction gap of 60 days or more.

Business Insight:

These accounts show low customer activity and may require attention.

Recommendation:

The bank should contact inactive customers through offers or reminders to encourage regular transactions.

Overall Summary

This analysis identifies active, top-performing, low-performing, and dormant accounts. It helps understand customer transaction behavior and account performance.

Overall Business Insight

Customer transaction patterns help identify high-performing and dormant accounts. This supports better account monitoring and customer management.

Overall Recommendation

Regularly monitor customer transactions, support low-performing and dormant accounts, and maintain good services for active customers.

Task 3: Customer Profile Building

In this task, customer accounts are grouped based on their activity, balance, and transaction behaviour. This helps understand different types of customers.

Activity Level Rubric Definition

To segment customer accounts based on transaction frequency, the following rubric criteria have been applied:

- **High Activity:** Accounts with **more than 5 transactions** (`TransactionCount > 5`)
- **Medium Activity:** Accounts with **3 to 5 transactions** (`3 <= TransactionCount <= 5`)
- **Low Activity:** Accounts with **fewer than 3 transactions** (`TransactionCount < 3`)

```
# Count Transactions for Each Account

activity = df.groupby("AccountID").size().reset_index(name="TransactionCount")

# Create Activity Level

activity["ActivityLevel"] = activity["TransactionCount"].apply(
    lambda x: "High" if x > 5 else ("Medium" if x >= 3 else "Low")
)

activity.head()
```

	AccountID	TransactionCount	ActivityLevel
0	ACC10117	9	High
1	ACC10996	8	High
2	ACC11062	4	Medium
3	ACC11188	2	Low
4	ACC11285	5	Medium

Next steps: [Generate code with activity](#) [New interactive sheet](#)

```
# Number of Accounts in Each Activity Level

activity["ActivityLevel"].value_counts()
```

	count
ActivityLevel	
Medium	92
Low	49
High	48

dtype: int64

```
# Average Balance and Transaction Volume for Each Account

customer_segment = df.groupby("AccountID").agg({
    "AccountBalance": "mean",
    "TransactionAmount": "sum"
}).reset_index()

customer_segment.head()
```

	AccountID	AccountBalance	TransactionAmount	
0	ACC10117	94082.590727	599628.73770	
1	ACC10996	72517.761136	506746.15806	
2	ACC11062	73541.328302	247384.20924	
3	ACC11188	51359.039950	127634.92588	
4	ACC11285	79923.619870	169318.96953	

Next steps: [Generate code with customer_segment](#) [New interactive sheet](#)

Customer Segmentation

Observation:

The table shows the average account balance and total transaction amount for each customer account.

Business Insight:

Customers with higher account balances also have higher transaction amounts.

Recommendation:

The bank should provide personalized services, premium products, and loyalty programs for high-value customers while encouraging other customers to increase account activity.

```
# High Net Inflow Accounts

high_inflow = account_summary.sort_values("NetInflow", ascending=False).head(5)

high_inflow
```

TransactionType	Deposit	Withdrawal	NetInflow	
AccountID				
ACC48501	323846.55886	25215.946120	298630.612740	
ACC57700	250121.85850	0.000000	250121.858500	
ACC38559	219919.51623	24073.272256	195846.243974	
ACC83581	184645.18445	0.000000	184645.184450	
ACC18177	174360.26978	0.000000	174360.269780	

Next steps: [Generate code with high_inflow](#) [New interactive sheet](#)

Observation

The top 5 accounts have the highest positive net inflow.

Business Insight

These accounts received more money than they withdrew.

Recommendation

Provide good banking services and loyalty benefits to retain these customers..

```
# High Frequency Low Balance Accounts

profile1 = df.groupby("AccountID").agg({
    "TransactionAmount": "count",
    "AccountBalance": "mean"
}).reset_index()

profile1.columns = ["AccountID", "TransactionCount", "AverageBalance"]

high_freq_low_balance = profile1[
    (profile1["TransactionCount"] >= 5) &
    (profile1["AverageBalance"] < profile1["AverageBalance"].median())
]

high_freq_low_balance.head()
```

	AccountID	TransactionCount	AverageBalance	
1	ACC10996	8	72517.761136	
11	ACC15671	6	54565.019303	
14	ACC16664	10	68276.438402	
18	ACC18177	6	58248.054195	
32	ACC24508	7	57275.485464	



Next steps:

[Generate code with high_freq_low_balance](#)

[New interactive sheet](#)

High-Frequency Low-Balance Accounts

Observation:

The table shows accounts with many transactions but a low average balance.

Business Insight:

These customers use their accounts regularly but keep less money in them.

Recommendation:

Encourage customers to increase their account balance.

Negative or Near-Zero Balance Accounts

```
low_balance = df[df["AccountBalance"] <= 1000][
    ["AccountID", "AccountBalance"]
].drop_duplicates()

low_balance.head()
```

	AccountID	AccountBalance	
599	ACC11837	-467.139424	
350	ACC17688	-6998.284258	
391	ACC18177	-6158.757250	
507	ACC19156	-31233.114930	
158	ACC26940	-22668.046670	



Next steps:

[Generate code with low_balance](#)

[New interactive sheet](#)

Negative or Near-Zero Balance Accounts

Observation:

The table shows accounts with very low or near-zero balance.

Business Insight:

These accounts may need attention because they have very little money.

Recommendation:

Monitor these accounts and encourage customers to improve their account balance.

Overall Business Insight

This analysis groups customers based on account activity, balance, transaction amount, and net inflow. It helps identify different customer segments.

Overall Recommendation

Provide suitable banking services for different customer groups and encourage low-balance or inactive customers to improve their account activity.

Task 4: Financial Risk Identification

In this task, customer risk scores and credit ratings are analyzed to identify high-risk accounts. This helps the bank understand which customers may need more attention.

```
# Accounts with Frequent Large Withdrawals

frequent_large = df[
    (df["TransactionType"] == "Withdrawal") &
    (df["TransactionAmount"] > 80000)
]

frequent_large = frequent_large.groupby("AccountID").size().reset_index(name="LargeWithdrawalCount")

frequent_large = frequent_large[frequent_large["LargeWithdrawalCount"] >= 2]

frequent_large
```

	AccountID	LargeWithdrawalCount	
0	ACC10117	2	
4	ACC15671	2	
11	ACC29231	2	
14	ACC33287	2	

Next steps:

[Generate code with frequent_large](#)

[New interactive sheet](#)

Observation

ACC29231 and ACC33287 each have 2 large withdrawals.

Business Insight

These accounts show repeated high-value withdrawal activity.

Recommendation

Monitor these accounts and verify that the transactions are normal. Accounts with Frequent Large Withdrawals

```
# Balance Volatility using Standard Deviation

balance_volatility = df.groupby("AccountID")["AccountBalance"].std().reset_index()

balance_volatility.columns = ["AccountID", "BalanceStd"]

balance_volatility = balance_volatility.sort_values("BalanceStd", ascending=False)

balance_volatility.head()
```

	AccountID	BalanceStd	
185	ACC97411	68179.288932	
133	ACC70741	67364.539557	
83	ACC45101	64368.106460	
171	ACC89098	61651.692978	
82	ACC43771	59638.348003	

Next steps:

[Generate code with balance_volatility](#)

[New interactive sheet](#)

Balance Volatility

Observation:

The table shows accounts with the highest balance changes.

Business Insight:

These accounts have unstable account balances. They may need regular monitoring.

Recommendation:

The bank should review these accounts and understand the reason for large balance changes.

```
# Detect Anomalies using IQR

Q1 = df["TransactionAmount"].quantile(0.25)
Q3 = df["TransactionAmount"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
```

```
anomalies = df[
    (df["TransactionAmount"] < lower) |
    (df["TransactionAmount"] > upper)
]

anomalies[["AccountID", "TransactionAmount"]].head()
```

	AccountID	TransactionAmount
9	ACC19178	138133.82160
533	ACC23736	141210.80340
161	ACC35419	-36173.69555
370	ACC77533	154662.04580
525	ACC92104	144137.88030

Transaction Amount Anomalies

Observation:

The table shows unusual transaction amounts.

Business Insight:

These transactions are different from normal transactions and may need further checking.

Recommendation:

The bank should review these transactions to reduce financial risk.

```
# Suspicious Customers

suspicious = df[
    (df["RiskScore"] > 0.7) &
    (df["TransactionAmount"] > df["TransactionAmount"].quantile(0.90))
][["CustomerID", "AccountID", "TransactionAmount", "RiskScore"]]

suspicious.head()
```

	CustomerID	AccountID	TransactionAmount	RiskScore
98	CUST3315	ACC10996	92094.28876	0.783476
78	CUST5920	ACC10996	116828.26160	0.925928
327	CUST4402	ACC19178	92902.53666	0.895799
47	CUST1497	ACC21264	106306.42830	0.794521
533	CUST7098	ACC23736	141210.80340	0.885864

Next steps:

[Generate code with suspicious](#)

[New interactive sheet](#)

Suspicious Customers

Observation:

The table shows customers with high risk scores and large transactions.

Business Insight:

These customers may have unusual transaction behaviour.

Recommendation:

The bank should monitor these customers and verify suspicious transactions.

Overall Summary

This task analyzed customer transactions to identify financial risks. It detected accounts with frequent large withdrawals, unstable balances, unusual transaction amounts, and suspicious customer behaviour. These findings help the bank identify potential risks early and improve financial monitoring.

Overall Business Insight:

The risk analysis helps identify risky accounts, unusual transactions, and unstable account balances. This helps the bank improve risk management.

Overall Recommendation

The bank should regularly monitor high-risk accounts, investigate unusual transactions, and update customer risk profiles. Early identification and timely action can reduce financial losses and improve overall risk management.

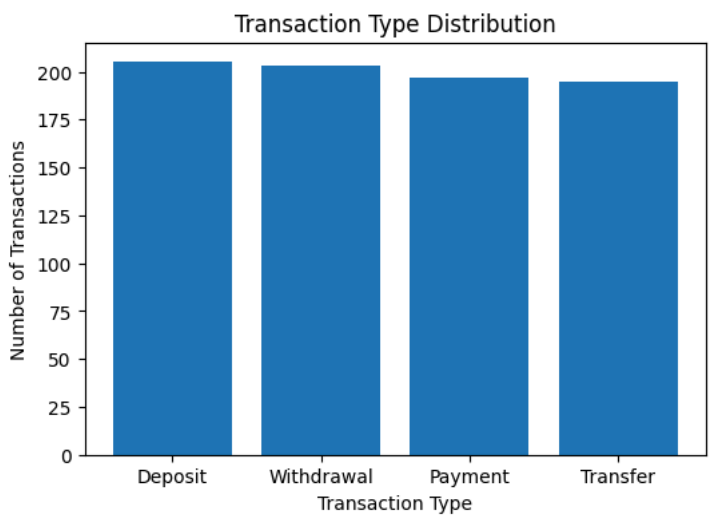
Task 5: Visualization

In this task, different charts are created to explore customer transactions, account balances, risk scores, and customer behaviour. These visualizations help identify trends, compare different customer groups, and support better business decisions.

Visualization 1: Transaction Type Distribution

```
transaction_count = df["TransactionType"].value_counts()

plt.figure(figsize=(6,4))
plt.bar(transaction_count.index, transaction_count.values)
plt.title("Transaction Type Distribution")
plt.xlabel("Transaction Type")
plt.ylabel("Number of Transactions")
plt.show()
```



Observation

The chart shows the number of transactions for each transaction type.

Business Insight

All transaction types have a similar transaction count, indicating balanced customer transaction behaviour.

Recommendation

The bank should continue monitoring all transaction types to identify future changes in customer transaction patterns.

Visualization 2: Total Transaction Amount by Product

```
# Total Transaction Amount by Product

product_amount = df.groupby("Product")["TransactionAmount"].sum()

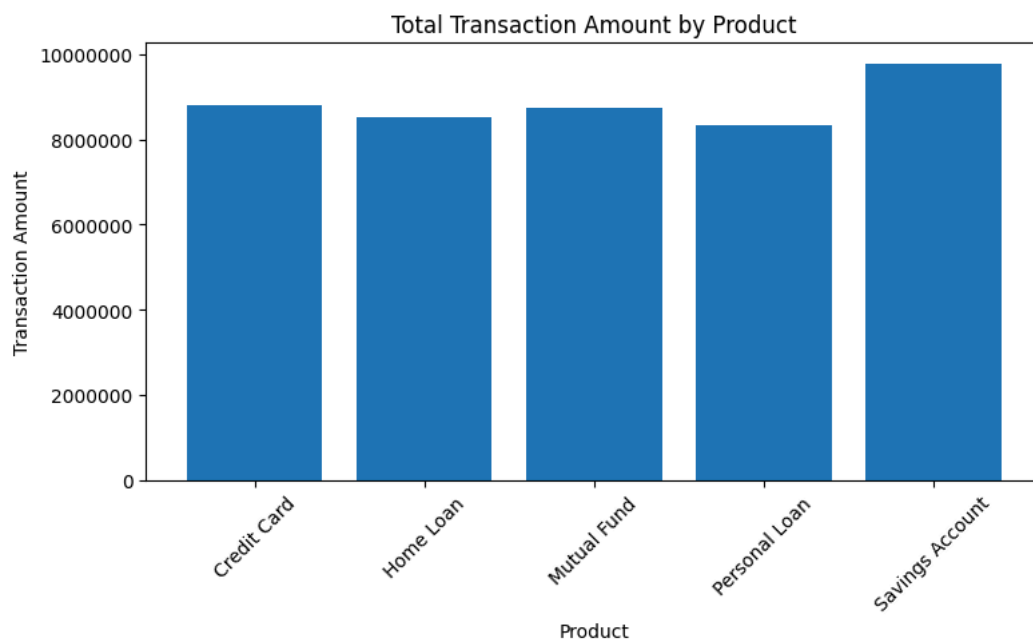
plt.figure(figsize=(8,5))

plt.bar(product_amount.index, product_amount.values)

plt.title("Total Transaction Amount by Product")
plt.xlabel("Product")
plt.ylabel("Transaction Amount")

plt.xticks(rotation=45)
plt.ticklabel_format(style='plain', axis='y')
plt.tight_layout()

plt.show()
```



Observation:

The chart shows the total transaction amount for each banking product.

Business Insight:

Some products have higher transaction amounts than others. These products contribute more to overall banking activity and customer engagement.

Recommendation:

The bank should promote high-performing products and improve the performance of products with lower transaction amounts.

Visualization 3: Average Account Balance by Account Type

```
# Average Account Balance by Account Type

balance_type = df.groupby("AccountType")["AccountBalance"].mean()

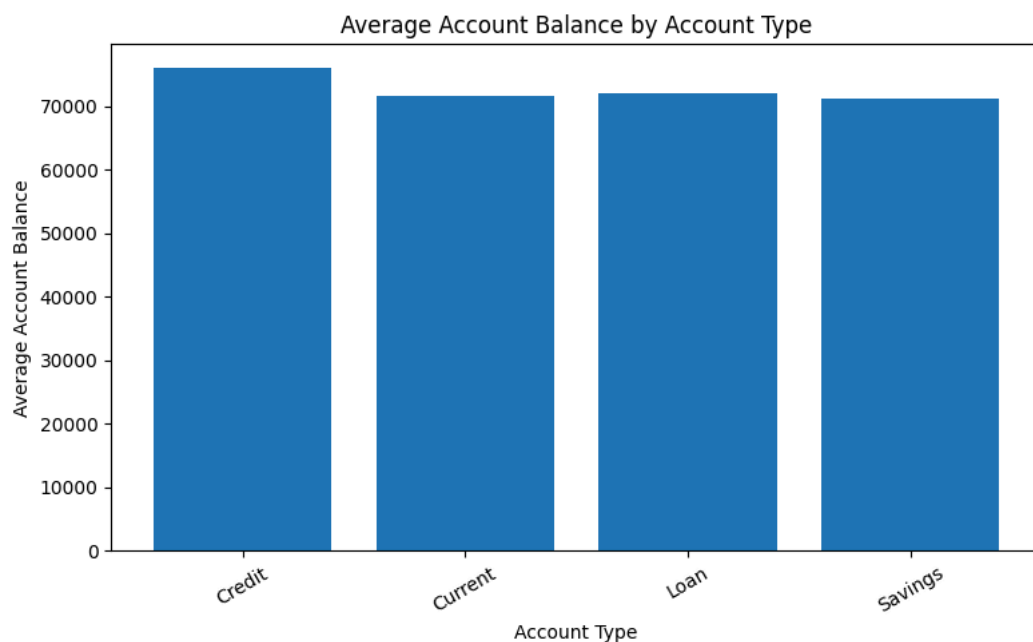
plt.figure(figsize=(8,5))

plt.bar(balance_type.index, balance_type.values)

plt.title("Average Account Balance by Account Type")
plt.xlabel("Account Type")
plt.ylabel("Average Account Balance")

plt.xticks(rotation=30)
plt.ticklabel_format(style='plain', axis='y')

plt.tight_layout()
plt.show()
```



Observation

The chart shows the total transaction amount for each banking product. Savings Account has the highest transaction amount, while Personal Loan has the lowest.

Business Insight

Savings Account contributes the most to overall banking transactions, indicating high customer usage. Personal Loan has comparatively lower transaction activity, while other products show similar transaction amounts.

Recommendation

The bank should continue strengthening Savings Account services and promote lower-performing products such as Personal Loan through targeted marketing, attractive offers, and customer awareness programs.

```
# Monthly Transaction Trend

import matplotlib.pyplot as plt

# Convert TransactionDate to datetime
df['TransactionDate'] = pd.to_datetime(df['TransactionDate'])

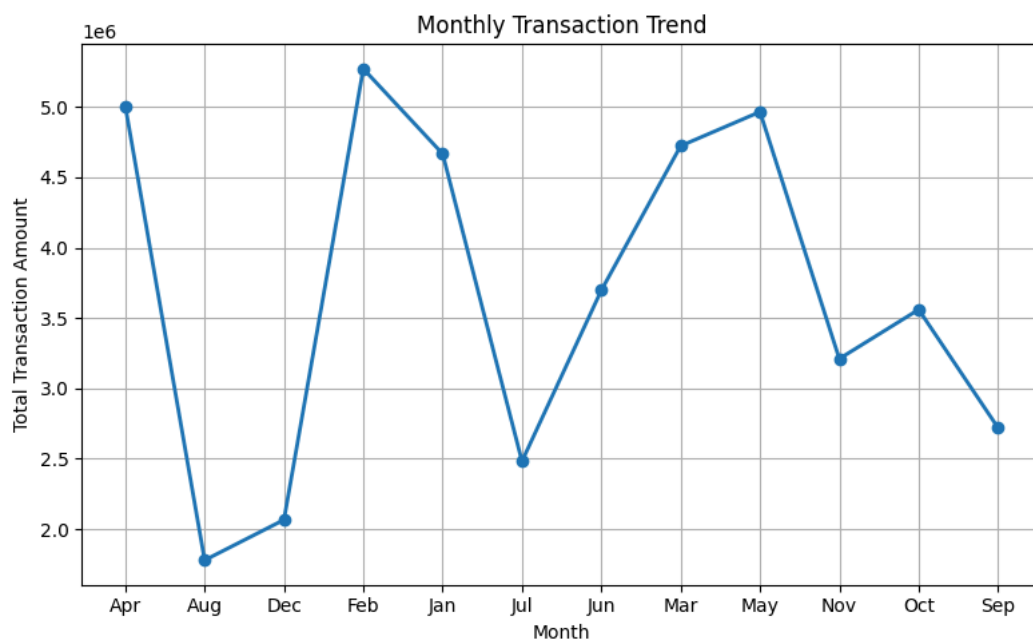
# Create Month column
df['Month'] = df['TransactionDate'].dt.strftime('%b')

# Monthly total transaction amount
monthly_trend = df.groupby('Month')['TransactionAmount'].sum()

# Plot
plt.figure(figsize=(8,5))
plt.plot(monthly_trend.index,
         monthly_trend.values,
         marker='o',
         linewidth=2)

plt.title('Monthly Transaction Trend')
plt.xlabel('Month')
plt.ylabel('Total Transaction Amount')
plt.grid(True)

plt.tight_layout()
plt.show()
```



Observation

The line chart shows the monthly trend of total transaction amounts. February has the highest transaction amount, followed by April and May, while August has the lowest transaction amount.

Business Insight

Higher transaction amounts in February, April, and May indicate increased customer activity during these months. Lower transaction activity in August suggests reduced customer engagement or seasonal variation.

Recommendation

The bank should analyze the reasons behind high-performing months and apply similar marketing strategies during low-transaction months such as August to improve customer engagement and transaction volume.

```
# Convert date
df["TransactionDate"] = pd.to_datetime(df["TransactionDate"])

# Month column
df["Month"] = df["TransactionDate"].dt.strftime("%b")

# Create trend table
trend = df.pivot_table(
    index="Month",
    columns="TransactionType",
    values="TransactionAmount",
    aggfunc="sum"
)
```

```
month_order = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
               'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

trend = trend.reindex(month_order).fillna(0)
```

```
import matplotlib.pyplot as plt

# Month order
month_order = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
               'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

# Reorder months
trend = trend.reindex(month_order).fillna(0)

plt.figure(figsize=(8,5))

plt.plot(
    trend.index,
    trend['Deposit'].values,
    marker='o',
    linewidth=2,
    label='Deposit'
)

plt.plot(
    trend.index,
```

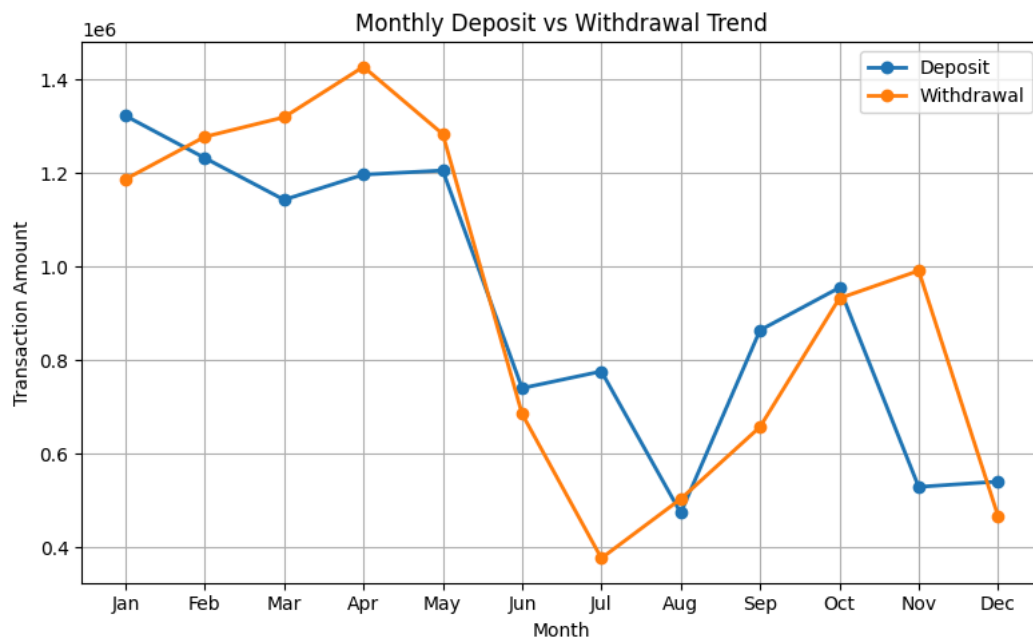
```

trend['Withdrawal'].values,
marker='o',
linewidth=2,
label='Withdrawal'
)

plt.title('Monthly Deposit vs Withdrawal Trend')
plt.xlabel('Month')
plt.ylabel('Transaction Amount')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

```



Observation

The line chart compares monthly Deposit and Withdrawal transaction amounts. Deposits are highest in January, while Withdrawals peak in April. Both transaction types decline during June to August, and their values are almost equal in October.

Business Insight

The monthly variation in deposits and withdrawals indicates changing customer cash flow patterns. Higher withdrawals in April and November may reflect increased customer spending, while higher deposits in January suggest stronger fund inflows.

Recommendation

The bank should monitor months with higher withdrawals and encourage customers to increase deposits through savings schemes, promotional offers, and financial planning services during low-deposit periods.

Visualization 6: Correlation Heatmap of Financial Variables

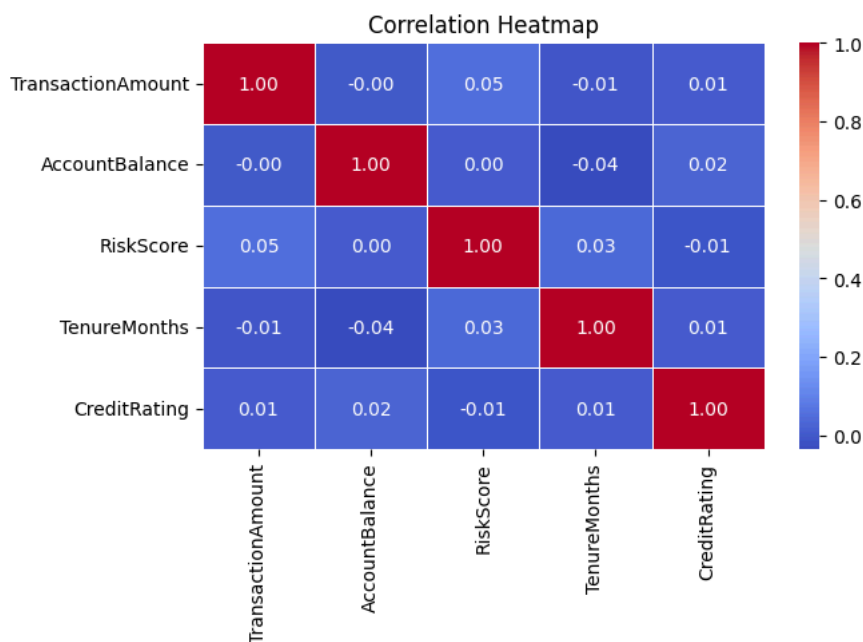
```
# Correlation Heatmap

import seaborn as sns
import matplotlib.pyplot as plt

# Select numerical columns
corr = df[['TransactionAmount',
          'AccountBalance',
          'RiskScore',
          'TenureMonths',
          'CreditRating']].corr()

# Plot Heatmap
plt.figure(figsize=(7,5))
sns.heatmap(corr,
            annot=True,
            cmap='coolwarm',
            fmt='.2f',
            linewidths=0.5)

plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()
```



Observation

The heatmap shows the correlation between Transaction Amount, Account Balance, Risk Score, Tenure Months, and Credit Rating. Most correlation values are close to zero, indicating weak relationships among these variables.

Business Insight

The weak correlations suggest that no single financial variable strongly influences another. Customer behaviour is affected by multiple factors rather than one variable alone.

Recommendation

The bank should combine multiple customer attributes instead of relying on a single factor while making business decisions, customer segmentation, and risk assessment.

Overall Observation

The visualizations show customer transaction behaviour, product performance, monthly transaction trends, and relationships among financial variables. They help identify customer patterns and business opportunities.

Overall Business Insight

The analysis highlights high-performing products, seasonal transaction trends, customer behaviour, and weak relationships among financial variables. These insights support better customer engagement and financial planning.

Overall Recommendation

The bank should regularly monitor transaction trends, strengthen high-performing products, improve low-performing services, and use data-driven strategies for customer segmentation and risk management.

Double-click (or enter) to edit

Task 6: Hypothesis Testing

Introduction

In this task, hypothesis testing is used to check whether customers with more transactions have higher average account balances than customers with fewer transactions. This helps the bank understand customer behavior using statistical analysis.

```
from scipy.stats import ttest_ind

# Count transactions for each account
transaction_count = df.groupby("AccountID").size().reset_index(name="TransactionCount")

# Average balance for each account
avg_balance = df.groupby("AccountID")["AccountBalance"].mean().reset_index()

# Merge both tables
hypothesis_df = transaction_count.merge(avg_balance, on="AccountID")

# Split into High and Low transaction groups
median_count = hypothesis_df["TransactionCount"].median()

high_volume = hypothesis_df[hypothesis_df["TransactionCount"] > median_count]["AccountBalance"]
low_volume = hypothesis_df[hypothesis_df["TransactionCount"] <= median_count]["AccountBalance"]
```

```
t_stat, p_value = ttest_ind(high_volume, low_volume, equal_var=False)

print("T-statistic:", t_stat)
print("P-value:", p_value)
```

T-statistic: 1.7595094127904853
P-value: 0.08013695745786936

```
if p_value < 0.05:
    print("Reject the Null Hypothesis")
else:
    print("Fail to Reject the Null Hypothesis")
```

Fail to Reject the Null Hypothesis

Hypothesis Test Result

Null Hypothesis (H_0): High-volume transaction accounts do not have a significantly higher average account balance than low-volume transaction accounts.

Alternative Hypothesis (H_1): High-volume transaction accounts have a significantly higher average account balance than low-volume transaction accounts.

Result: The p-value (0.0801) is greater than 0.05. Therefore, we fail to reject the null hypothesis.

Conclusion: There is not enough statistical evidence to conclude that high-volume transaction accounts have higher average balances than low-volume transaction accounts.

Business Insight: The hypothesis test indicates that transaction frequency alone is not a reliable indicator of higher account balances. Customer value should be assessed using multiple financial and behavioural factors.

Final Conclusion

Final Conclusion

This project analyzed customer transactions, account balances, customer segmentation, financial risks, and business trends using Python.

Data cleaning, exploratory data analysis, customer segmentation, financial risk identification, visualization, and hypothesis testing were successfully completed.